Unigram Tokenization

The Unigram algorithm is used with SentencePiece, which is applied in models like ALBERT, T5, mBART, Big Bird, and XLNet. SentencePiece solves the problem that not all languages use spaces to separate words. It treats the entire input as a raw input stream, where spaces are also considered as characters. Then it uses the Unigram algorithm to build an appropriate vocabulary. This section explains Unigram in depth, even showing a full implementation. If you only need a general overview, you can skip to the end.

Training Algorithm

Compared to BPE and WordPiece, Unigram works in the opposite direction. It starts with a large vocabulary and gradually removes tokens until it reaches the desired vocabulary size. There are several ways to build the initial vocabulary, such as taking the most common substrings from pre-tokenized words or applying BPE on the initial corpus with a large vocabulary size. At each training step, the Unigram algorithm computes a loss over the entire corpus. Then, for each symbol in the vocabulary, it calculates how much the loss would increase if that symbol were removed. The symbols that cause the smallest increase in loss are considered less important and are removed first. Since this process is computationally expensive, instead of removing one symbol at a time, a percentage p (a hyperparameter, usually 10 or 20) of symbols with the lowest impact on loss are removed together. This process is repeated until the desired vocabulary size is reached. However, base characters are never removed to ensure any word can still be tokenized.

This may still seem a bit unclear, because the main idea is to compute the loss and see how it changes when tokens are removed, but we have not yet explained how this is done. This step depends on the tokenization algorithm of the Unigram model, which we will now explore.

We reuse the previous example corpus: ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5), and take all substrings as the initial vocabulary: ["h", "u", "g", "hu", "ug", "p", "pu", "n", "un", "b", "bu", "s", "hug", "gs", "ugs"]

Tokenization Algorithm

A Unigram model is a type of language model where each token is independent, meaning it does not depend on previous tokens. The probability of a token X is simply its own probability. So, if we used this model to generate text, it would always prefer the most common tokens.

The probability of a token = its frequency divided by the total frequency of all tokens. For example, the token "ug" appears in "hug", "pug", and "hugs", so its frequency is 20. The frequencies of all subwords are: ("h", 15) ("u", 36) ("g", 20) ("hu", 15) ("ug", 20) ("p", 17) ("pu",

17) ("n", 16) ("un", 16) ("b", 4) ("bu", 4) ("s", 5) ("hug", 15) ("gs", 5) ("ugs", 5). The total frequency is 210, so the probability of "ug" is 20/210.

To tokenize a word, we consider all possible segmentations and compute the probability of each. Since tokens are independent, the total probability is the product of the probabilities of each token. For example, for "pug": ["p","u","g"] → probability = 5/210 × 36/210 × 20/210 = 0.000389. ["pu","g"] → probability = 5/210 × 20/210 = 0.0022676, which is higher. So, segmentations with fewer tokens usually have higher probability.

All possible segmentations of "pug": ["p","u","g"] : 0.000389, ["p","ug"] : 0.0022676, ["pu","g"] : 0.0022676. Therefore, the final tokenization will be ["p","ug"] or ["pu","g"] (whichever appears first).

Viterbi Algorithm

Finding all possible segmentations is not always easy, so the Viterbi algorithm is used. It builds a graph where there is a branch from character a to b if the substring exists in the vocabulary, and assigns a probability to that branch. The Viterbi algorithm finds the best scoring segmentation at each position. By moving from the beginning to the end, it determines the best path and produces the final tokenization.

Example for "unhug": Character 0: "u" (0.171429), Character 1: "un" (0.076191), Character 2: "un h" (0.005442), Character 3: "un hu" (0.005442), Character 4: "un hug" (0.005442). So the tokenization is ["un","hug"].

Back to Training (Loss Calculation)

During training, each word is tokenized and its score is computed. Then the loss is calculated using negative log likelihood, which is the sum of -log(P(word)) for all words. Example: "hug": ["hug"] (0.071428), "pug": ["pu","g"] (0.007710), "pun": ["pu","n"] (0.006168), "bun": ["bu","n"] (0.001451), "hugs": ["hug","s"] (0.001701).

Loss = 10×(-log(0.071428)) + 5×(-log(0.007710)) + 12×(-log(0.006168)) + 4×(-log(0.001451)) + 5×(-log(0.001701)) = 169.8

Token Removal Effect

Now we check how removing each token affects the loss. For example, removing "pu" does not change the loss, because "pug" can be tokenized differently with the same score. But removing "hug" makes things worse: "hug" → ["hu","g"] and "hugs" → ["hu","gs"], which increases the loss by 23.5. Therefore, "pu" will likely be removed, but "hug" will be kept.